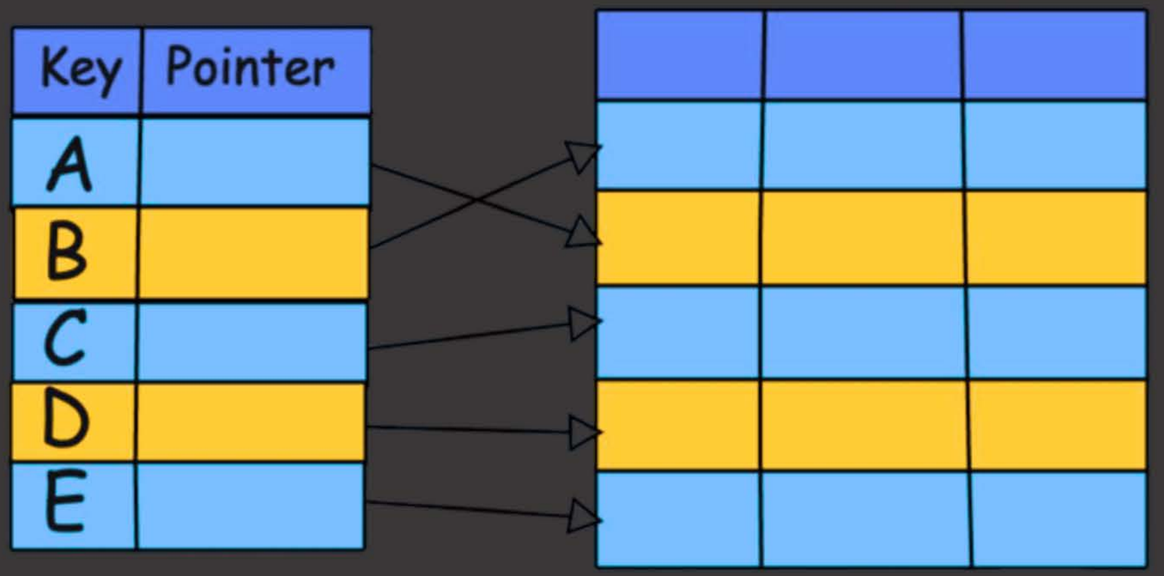
CSE370: Database Systems

Lecture 7: Database Indexing



Indexing

- Many queries reference only a small proportion of the records in a table. It is inefficient for the system to read every tuple in the instructor table to check if the DeptName value is “Physics” or find one instructor with ID “76766”. For a huge database, the system will perform poorly during search queries/table joins.

- To speed up data access, additional data structures called Indexes are created. Indexes ensure logarithmic time complexity even for searching through unsorted data.

- Index files are much smaller than the original data files stored on disk. As disk I/O is slower, the smaller index file is stored in memory, which has limited space but allows faster data access.

- Indexes can be created on any attribute that are used frequently in search queries/table joins to speed up data access. Insertion/Deletion/Update performance becomes slower as data has to be also inserted/deleted/updated in the tables as well as the indexes.

Instructor

ID	Name	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Types of Indexes

Primary Index

- Is also called clustering index.
- The table is physically ordered (or sorted) by the primary index search key.
- There can only be 1 primary index per table and it is usually the primary key (but not always)

Secondary Index

- Is also called non-clustering index.
- The table is not ordered (or sorted) by the secondary index search key.
- There can be multiple secondary indexes per table.

Examples of Indexes (Primary & Secondary)

ID and Department attributes are indexes in both tables below. In Table 1, ID is the primary index and Department is the secondary index. However, in Table 2, ID is the secondary index and Department is the primary index as the table is sorted by the search key values of the Department attribute.

ID	Name	Department
1	.....	EEE
2	.....	CSE
3	.....	BBS
4	.....	CSE

Table 1

ID	Name	Department
3	.....	BBS
2	.....	CSE
4	.....	CSE
1	.....	EEE

Table 2

Types of Index Files

Index files consists of records (called index entries) in the form shown on the right, where search-key is the value of an attribute or set of attributes used to look up records and pointer is the memory address of the original data stored in the disk.

Search-key	Pointer

Dense Index File

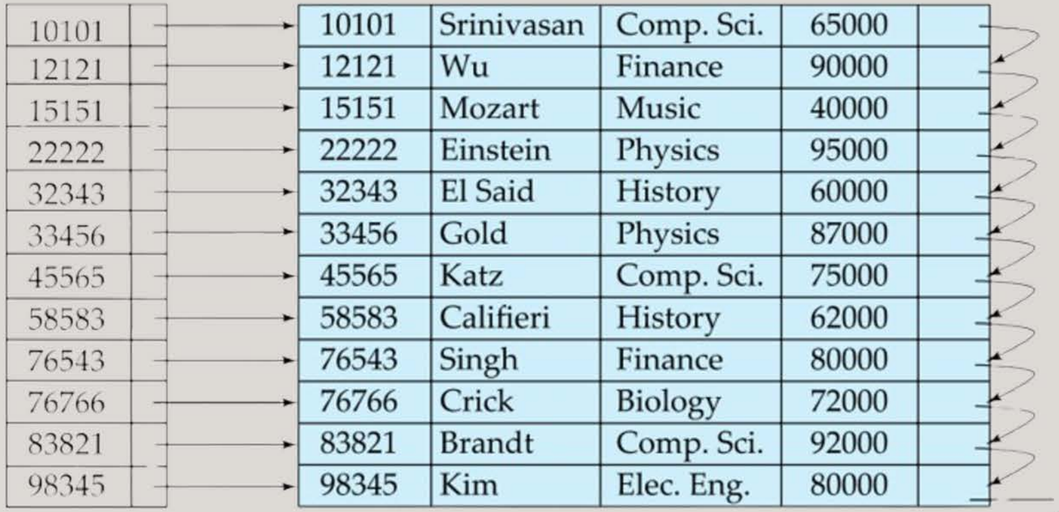
- Every single search key value has an entry in the index file.
- Can be used for both secondary and primary indexes.
- Requires more space (may not fit in main memory), but faster searching.
- Higher maintenance overhead for insertion/deletion.

Sparse Index File

- Only some search key values appear in the index files.
- Can be used only for primary indexes.
- Reduces the space requirement, but searching will be slower compared to Dense Files.
- Lower maintenance overhead for insertion/deletion operations.

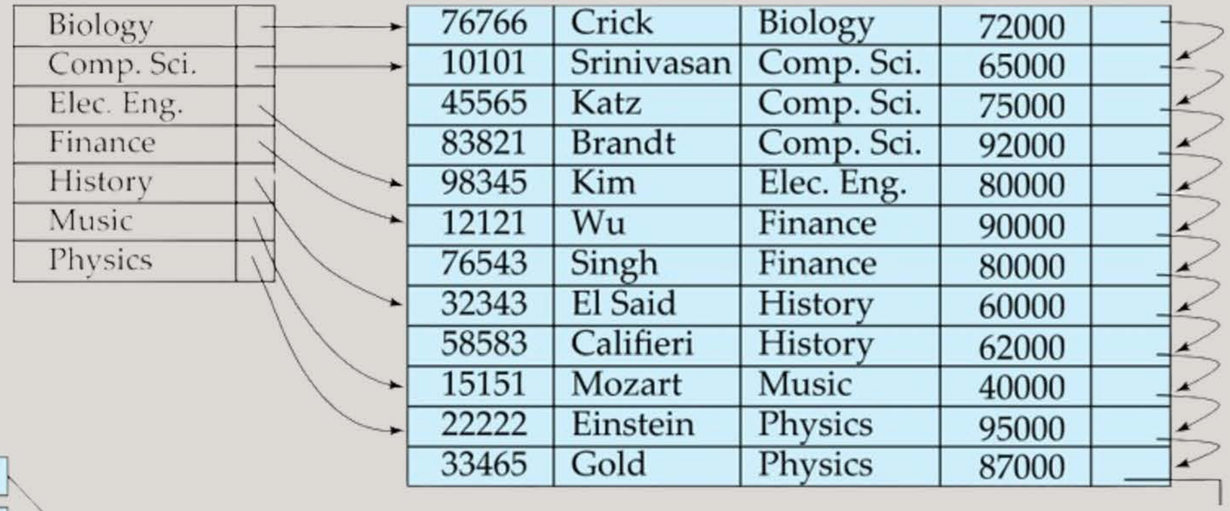
Examples of Index Files (Dense)

Figure 1 and 2, show dense index files on the primary index “ID” and “Department_Name” respectively. Figure 3 shows a dense index file on a secondary index “Salary”.



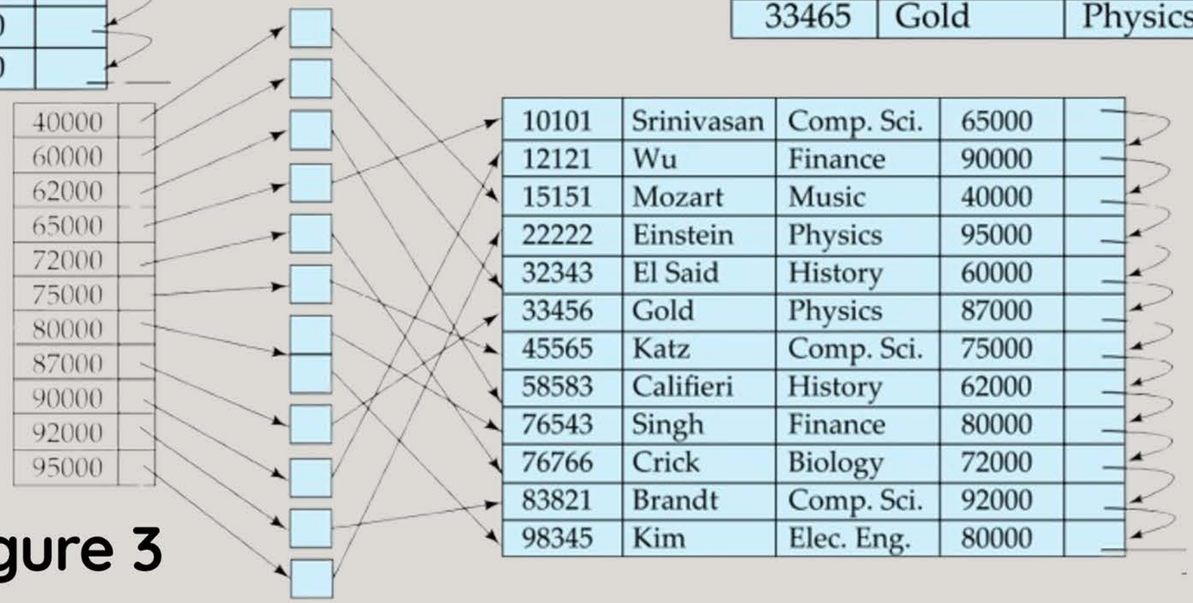
10101	10101	Srinivasan	Comp. Sci.	65000
12121	12121	Wu	Finance	90000
15151	15151	Mozart	Music	40000
22222	22222	Einstein	Physics	95000
32343	32343	El Said	History	60000
33456	33456	Gold	Physics	87000
45565	45565	Katz	Comp. Sci.	75000
58583	58583	Califieri	History	62000
76543	76543	Singh	Finance	80000
76766	76766	Crick	Biology	72000
83821	83821	Brandt	Comp. Sci.	92000
98345	98345	Kim	Elec. Eng.	80000

Figure 1



Biology	76766	Crick	Biology	72000
Comp. Sci.	10101	Srinivasan	Comp. Sci.	65000
Elec. Eng.	45565	Katz	Comp. Sci.	75000
Finance	83821	Brandt	Comp. Sci.	92000
History	98345	Kim	Elec. Eng.	80000
Music	12121	Wu	Finance	90000
Physics	76543	Singh	Finance	80000
	32343	El Said	History	60000
	58583	Califieri	History	62000
	15151	Mozart	Music	40000
	22222	Einstein	Physics	95000
	33465	Gold	Physics	87000

Figure 2

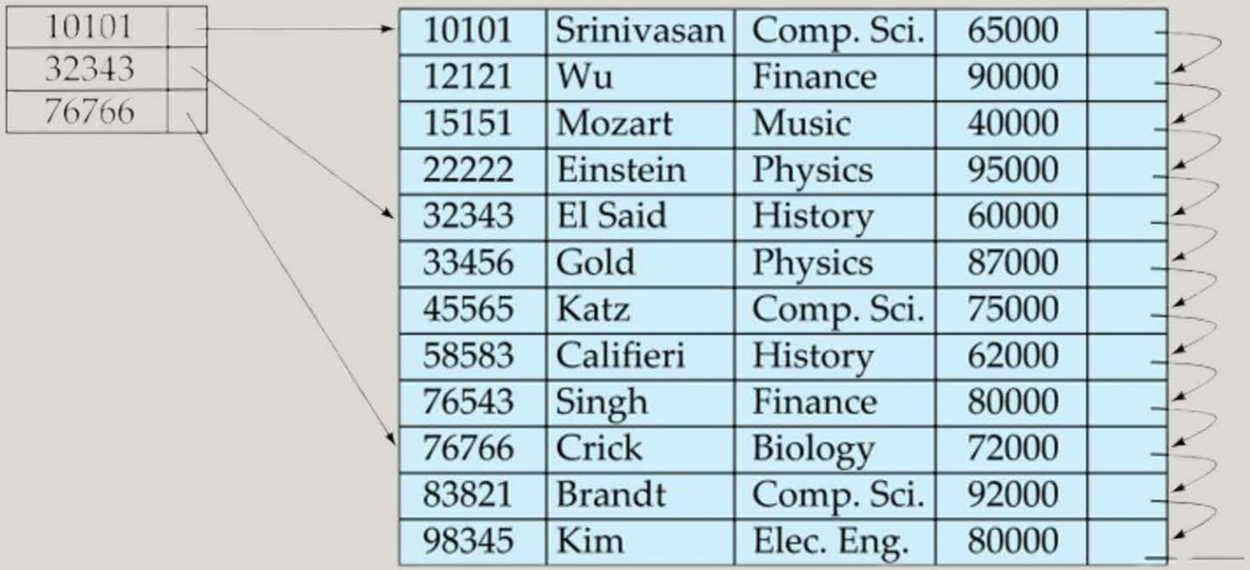


40000	10101	Srinivasan	Comp. Sci.	65000
60000	12121	Wu	Finance	90000
62000	15151	Mozart	Music	40000
65000	22222	Einstein	Physics	95000
72000	32343	El Said	History	60000
75000	33456	Gold	Physics	87000
80000	45565	Katz	Comp. Sci.	75000
87000	58583	Califieri	History	62000
90000	76543	Singh	Finance	80000
92000	76766	Crick	Biology	72000
95000	83821	Brandt	Comp. Sci.	92000
	98345	Kim	Elec. Eng.	80000

Figure 3

Examples of Index Files (Sparse)

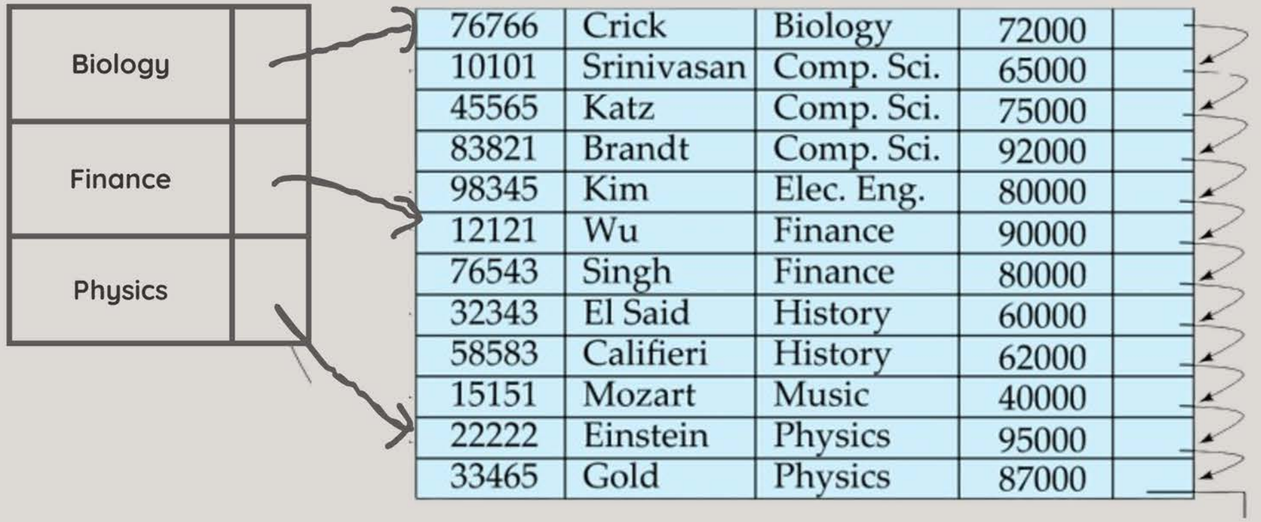
Figure 1 and 2, show sparse index files on the primary index “ID” and “Department_Name” respectively. Sparse files can only be created on primary indexes.



10101				
32343				
76766				

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

Figure 1



Biology				
Finance				
Physics				

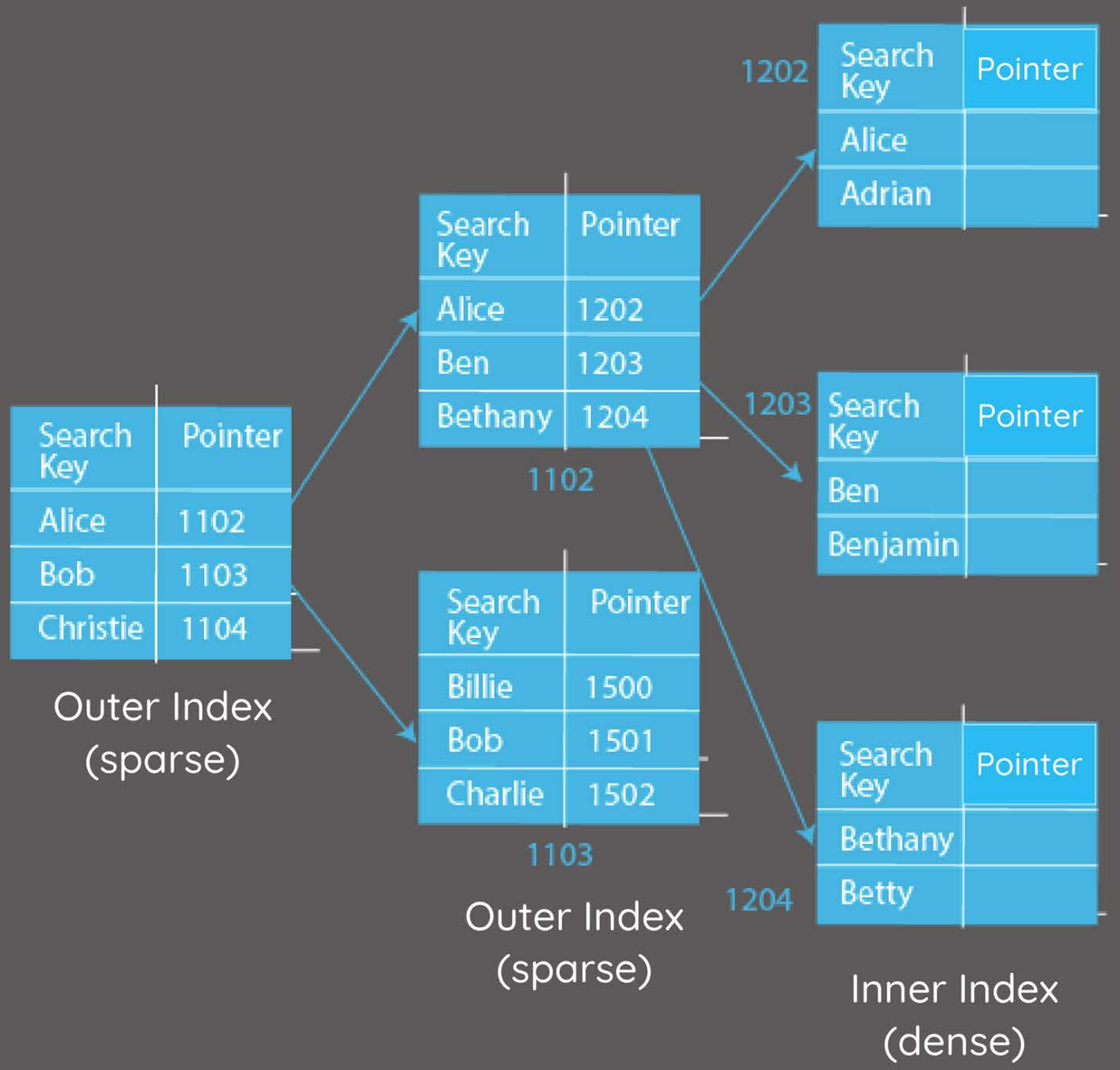
76766	Crick	Biology	72000	
10101	Srinivasan	Comp. Sci.	65000	
45565	Katz	Comp. Sci.	75000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	
12121	Wu	Finance	90000	
76543	Singh	Finance	80000	
32343	El Said	History	60000	
58583	Califieri	History	62000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
33465	Gold	Physics	87000	

Figure 2

Multilevel Indexing

Dense Index may not fit in main memory for large databases and accessing it directly from disk increases time complexity as disk I/O is more expensive. The solution is to create the index as a hierarchical structure in multiple levels instead of a single-level index.

One dense inner index is stored on disk and points to the original records in a table. One or more sparse outer index(es) are created on top of the inner index. The outer indexes can be stored in memory as they are smaller in size. This reduces the search space in the inner index and therefore, reduces disk I/O, thus significantly speeding up searches.



Data Structures for Indexing

There are a few data structures that are commonly used for different types of DBMS. For modern relational DBMS the most commonly used data structure for indexing is the B+ tree. Rarely, some RDBMS also use hash tables for indexes.

B+ tree is a type of multi-level index structure, where the root/internal nodes are sparse outer indexes and the leaf nodes make up the dense inner index. B+ tree are balanced trees ensuring logarithmic time complexity for searching any data.

For hash tables, searching might be done in constant time, however, it depends on the “goodness” of the hash function with respect to the search key. If the hash function does not guarantee uniform distribution of the search key, then in the worst case scenario every record might be searched to find any data.

B+ Tree Properties

- Every Tree has an order of “n”, which will determine the min/max number of children and keys in each node of the tree.
- All nodes on the left have smaller values than a key and all nodes on the right have greater than or equal values to a key.

	Max	Min
Children	root/internal: n leaf: 0	Root: 2 (if root is not leaf as well) Internal: ⌈n/2⌉
Keys/Values	n-1	Root: 1 Leaf and Internal: ⌈n/2⌉-1 [Note: for simplicity leaf and internal is considered same in some resources, otherwise leaf min is ⌈(n-1)/2⌉]

B+ Tree (Insertion)

- **Find the right spot (leaf node):** Start from the top (root) and move down the tree, following the correct path (based on the key you want to insert), until you reach a leaf node.

- **Insert if node has less than “max” keys:** If that leaf node has space (less than the max number of keys allowed), just insert the value in sorted order.

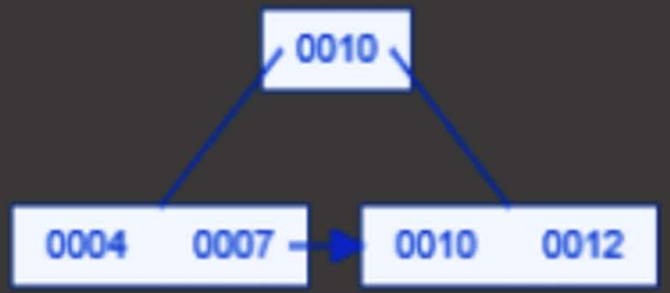
- **Split if node has “max” keys:** If the leaf is already full, split it into half. Put first half on a new left leaf and the second half on a new right leaf. Send up the lowest key from the new right node to the parent

- **Repeat the split upwards if needed:** If the parent now becomes full too, split it as well and push a key further up. If a non-leaf node is split, the lowest value on the right will only be moved up and not copied to the new right leaf. This might continue up to the root. If the root splits, a new root will be created and the tree height will increase.

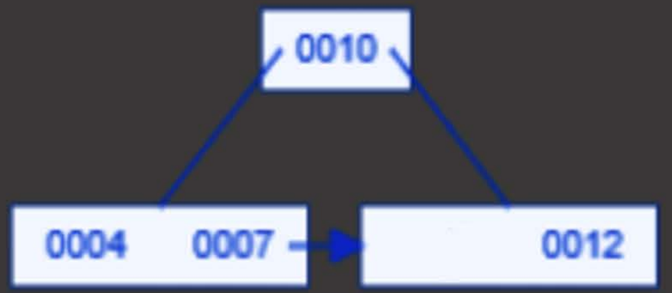
B+ Tree (Insertion)

Difference between splitting a leaf node vs a non-leaf(root/internal) node [for n= 4]

Leaf Node



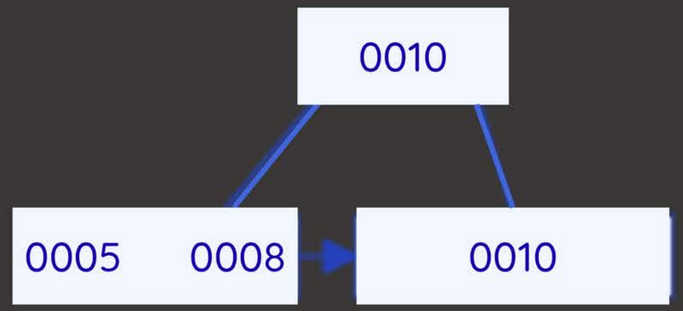
Non-Leaf Node



B+ Tree (Insertion)

If “n” is odd (e.g. n=3) then the splitting will result in either a left-biased or a right-biased tree. Right-biased means, the right node will contain 1 value more than the left node. Similarly for left-biased, the left node will contain 1 value more. Maintain the same bias for the entire tree.

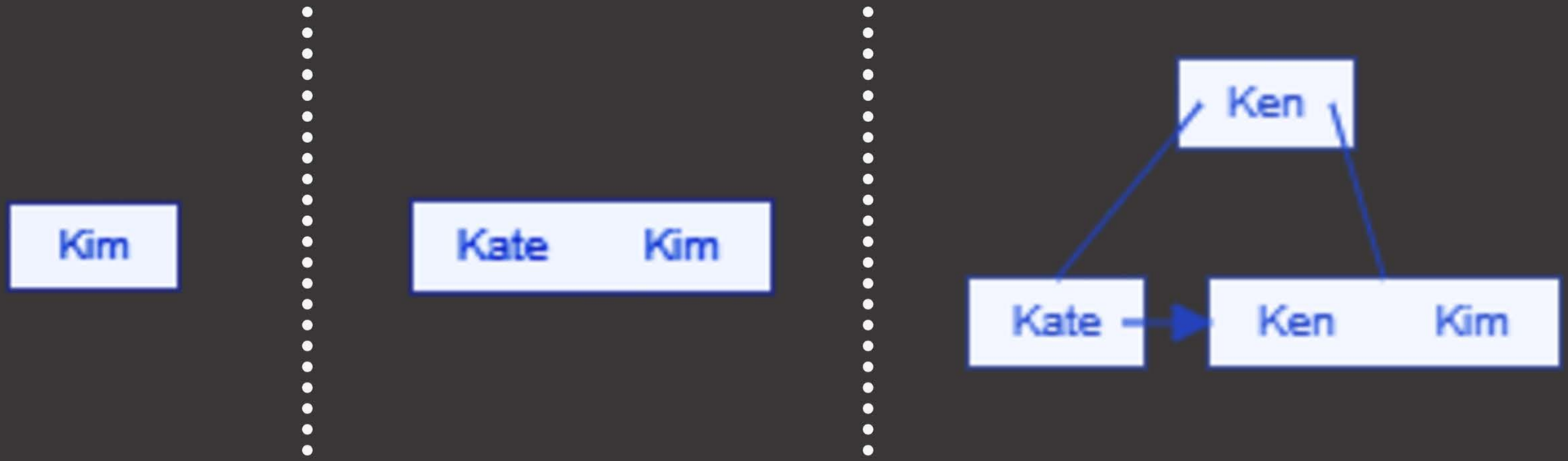
Left Bias



Right Bias

B+ Tree (Insertion)

If the keys have String values, then sorting should be done according to the “dictionary” order. Kim, Kate and Ken are inserted sequentially in the right-biased tree below [for n = 3].



B+ Tree (Insertion)

Insert the following in a B+ tree of order n = 4 according to the given sequence:

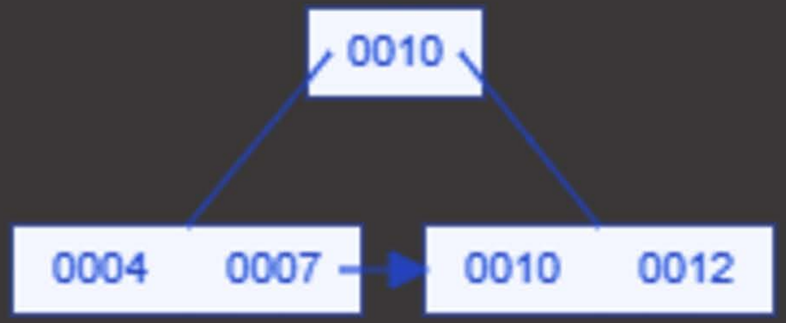
4, 10, 7, 12, 1, 5, 2, 25, 15, 9, 18, 20

Use the following B+ tree simulator to visualize the insertion process.

Here, max keys in a node = 4 - 1 = 3

Insert 4, 10, 7

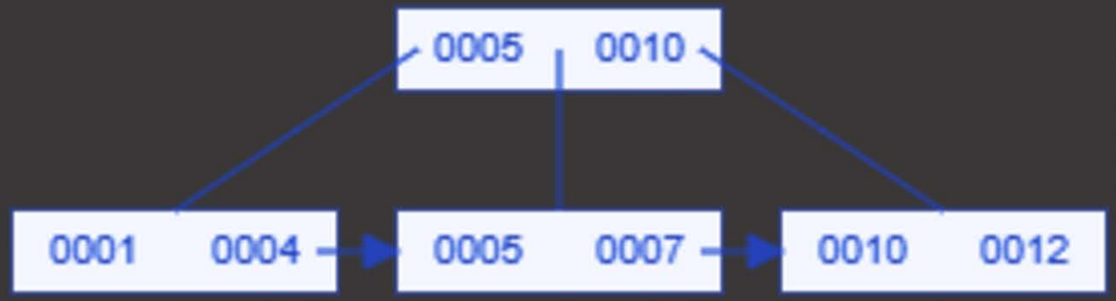
Insert 12



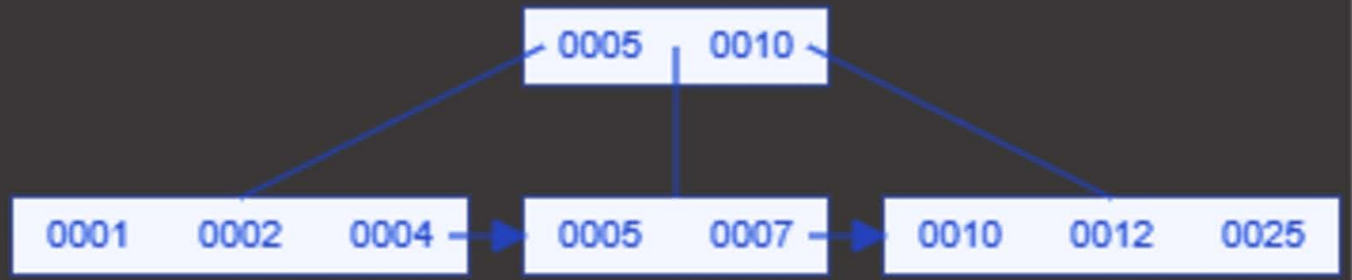
Insert 1

B+ Tree (Insertion)

Insert 5



Insert 2, 25



Insert 15

B+ Tree (Insertion)

Insert 9, 18

Insert 20

B+ Tree (Deletion)

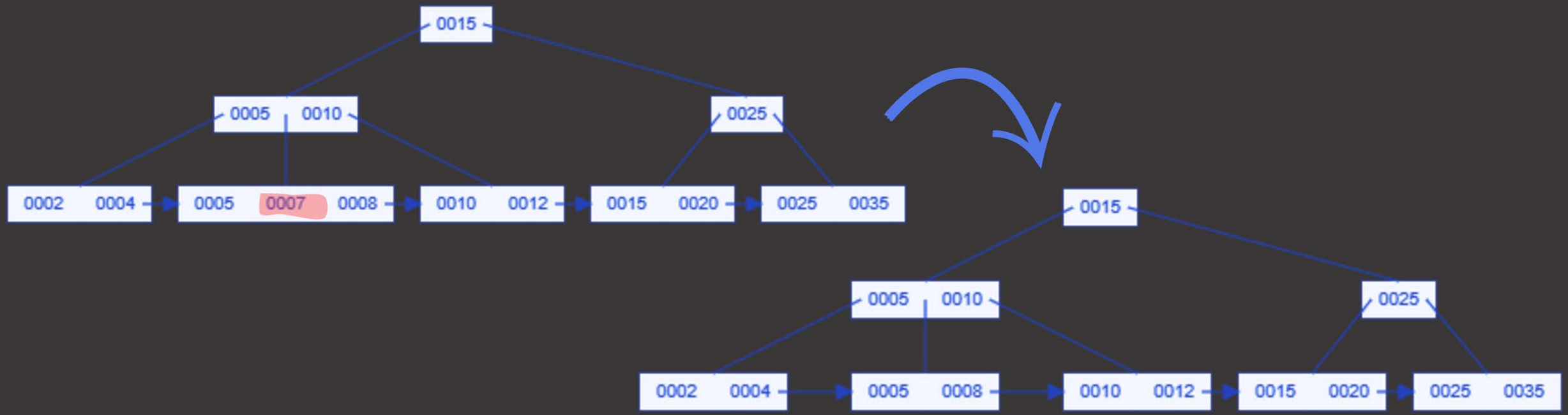
While deleting values from a B+ tree, if the value exists, then the target node will fall under the conditions of one of the 5 cases below. Use the following B+ tree simulator to visualize the deletion process.

- **CASE 1:** Number of keys > “Min” keys allowed
- **CASE 2:** Number of keys = “Min” keys allowed, but Sibling Nodes have > “Min” Keys allowed
- **CASE 3:** No. of Keys = “Min” Keys allowed, Sibling Nodes also have = “Min” Keys allowed, but Parent Node has > “Min” keys allowed.
- **CASE 4:** No. of Keys = “Min” Keys allowed, Sibling Nodes also have = “Min” Keys allowed, Parent Node also has = “Min” keys allowed (internal nodes).
- **CASE 5 (Special Case):** Same condition as Case 4. The special case is that the tree will shrink in height.

B+ Tree (Deletion - Case 1)

Number of keys in the target leaf node is greater than the “Min” number of keys allowed in node.

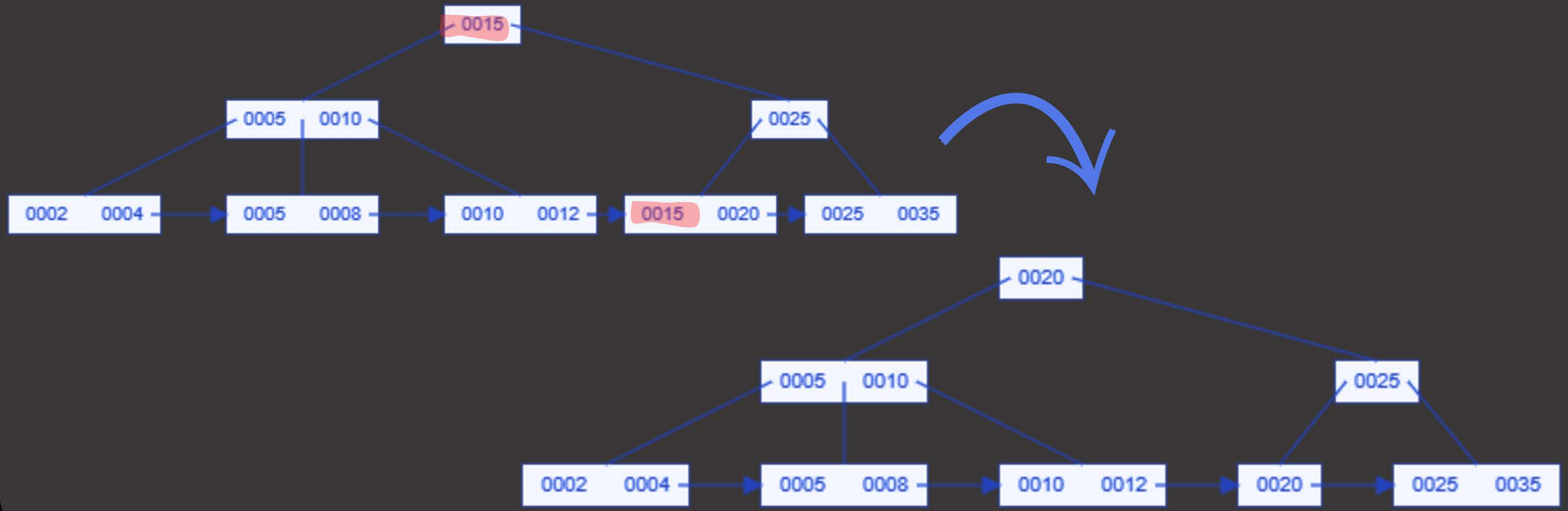
Case 1a (target value is only in leaf node): Delete 7 - For n=4, leaf/internal min keys = ⌈n/2⌉-1 = ⌈4/2⌉-1 = 1. Find key in leaf node and remove.



B+ Tree (Deletion - Case 1)

Number of keys in the target leaf node is greater than the “Min” number of keys allowed in node.

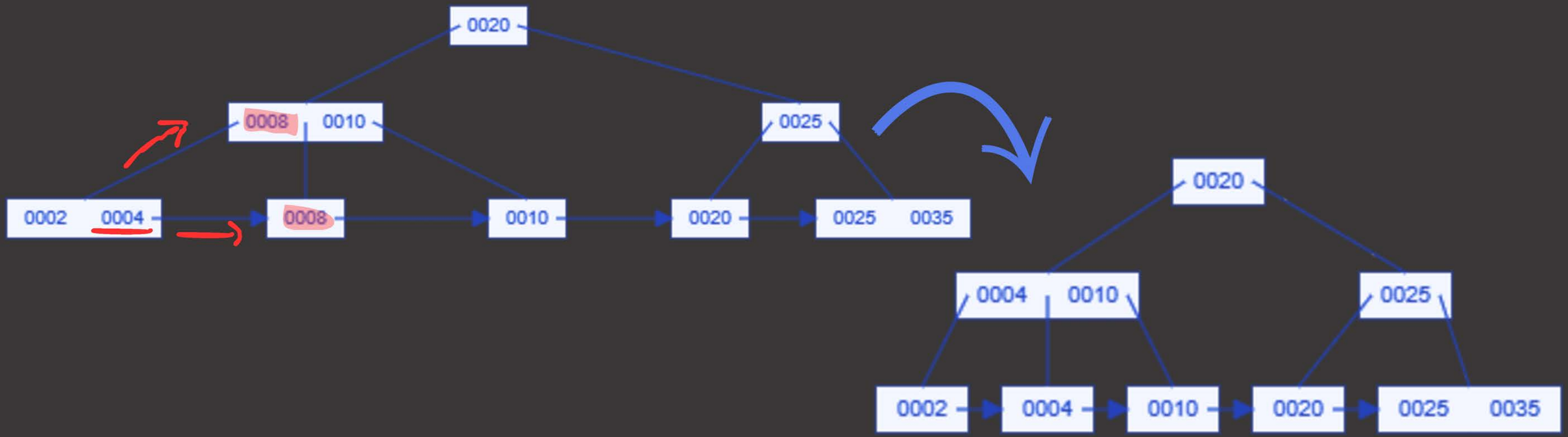
Case 1b (target value is in leaf node + internal node): Delete 15 - Find key in leaf node and remove. Then find key in internal node and replace it with lowest value from its right-most subtree



B+ Tree (Deletion - Case 2)

Number of keys in the target leaf node is equal to the “Min” number of keys allowed in node, however, at least one of the sibling nodes has more than minimum number of keys. Borrow from sibling.

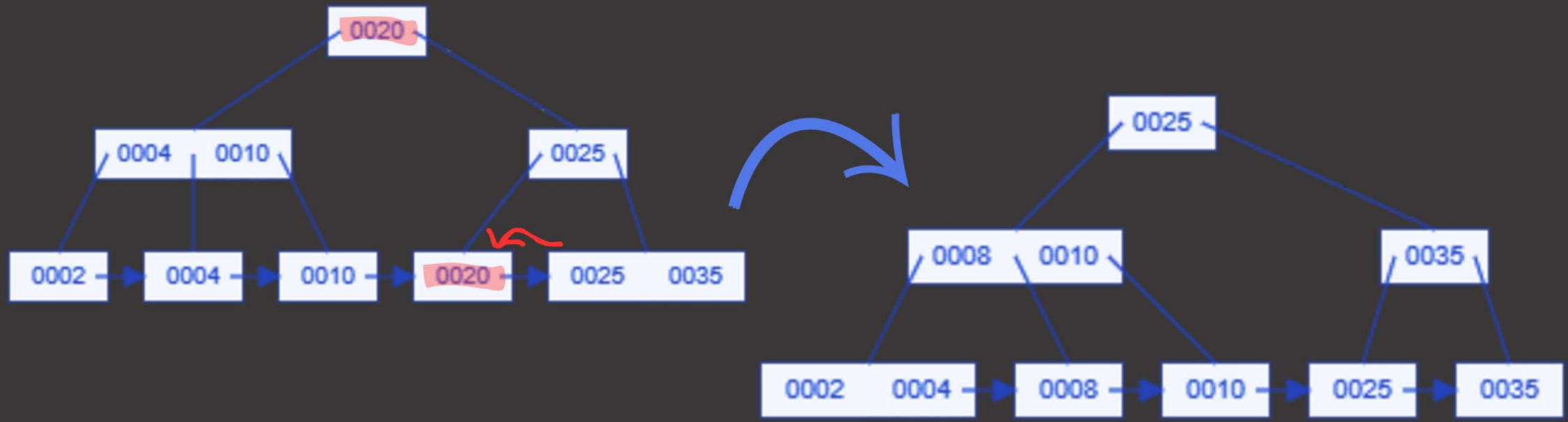
Case 2a (left sibling node > “min” keys allowed): Delete 8 - Find key in leaf node, the highest key from the left sibling node is copied to the parent and to the target node. For internal nodes, replace value with lowest value from right-most subtree.



B+ Tree (Deletion - Case 2)

Number of keys in the target leaf node is equal to the “Min” number of keys allowed in node, however, at least one of the sibling nodes has more than minimum number of keys. Borrow from sibling.

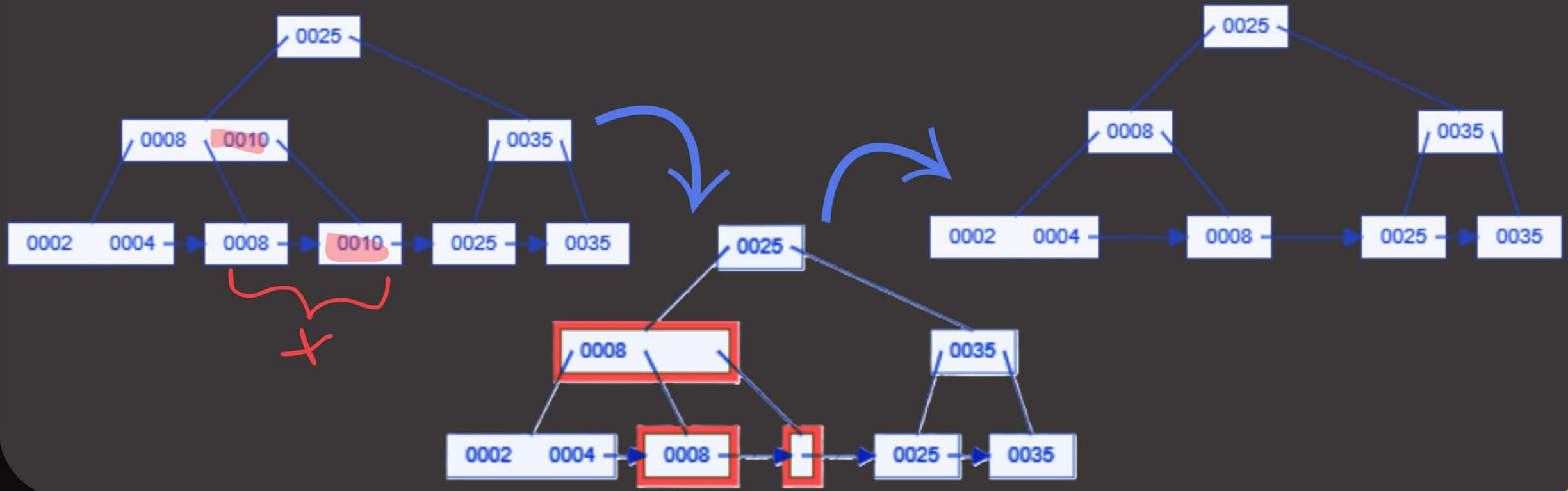
Case 2b (right sibling node > “min” keys allowed): Delete 20 - Find key in leaf node, the lowest key from the right sibling node is copied to the target node. The parent value is replaced with lowest value from right-most subtree. For internal nodes, replace value with lowest value from right-most subtree.



B+ Tree (Deletion - Case 3)

Number of keys in the target leaf node is equal to the “Min” number of keys allowed in node, same for sibling nodes. However, the parent node has more than minimum number of keys. Merge with sibling and remove parent key.

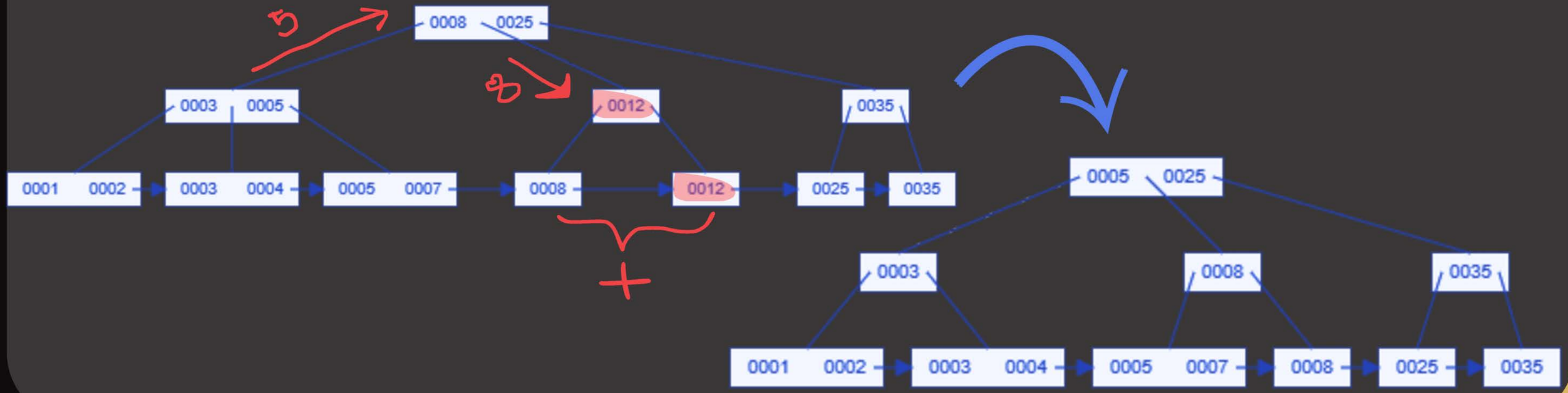
Case 3: Delete 10 - Find key in leaf node, remove key, merge the target node with its sibling node. Remove the parent key/value. For internal nodes, replace value with lowest value from right-most subtree.



B+ Tree (Deletion - Case 4)

Number of keys in the target leaf node is equal to the “Min” number of keys allowed in node, same for sibling and parent node. Parent node (which is an internal node) now will either borrow from it’s subling or merge with it’s sibling.

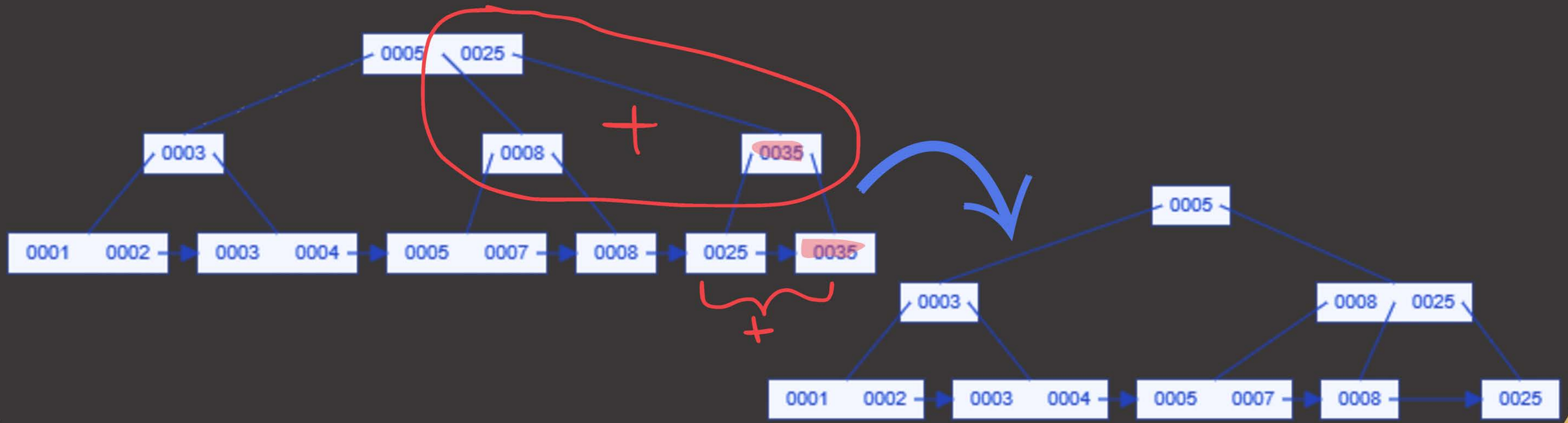
Case 4a (internal parent node borrows from sibling): Delete 12 - Find key in leaf node, remove and merge with sibling node like case 3. The internal parent node, will borrow 1 key from one of its sibling. The sibling will pass the key to it’s parent, and the parent will pass it’s old value to the internal node.



B+ Tree (Deletion - Case 4)

Number of keys in the target leaf node is equal to the “Min” number of keys allowed in node, same for sibling and parent node. Parent node (which is an internal node) now will either borrow from it’s subling or merge with it’s sibling.

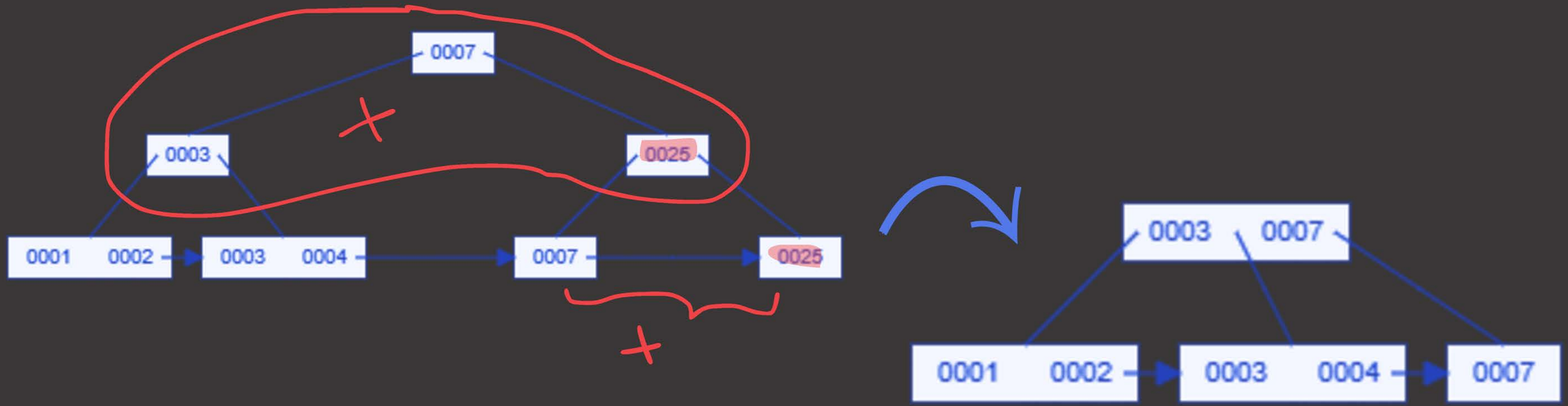
Case 4b (internal parent node merges with sibling): Delete 35 - Find key in leaf node, remove and merge with sibling node like case 3. The internal parent node, will merge with one of its sibling node and their parent key.



B+ Tree (Deletion - Case 5)

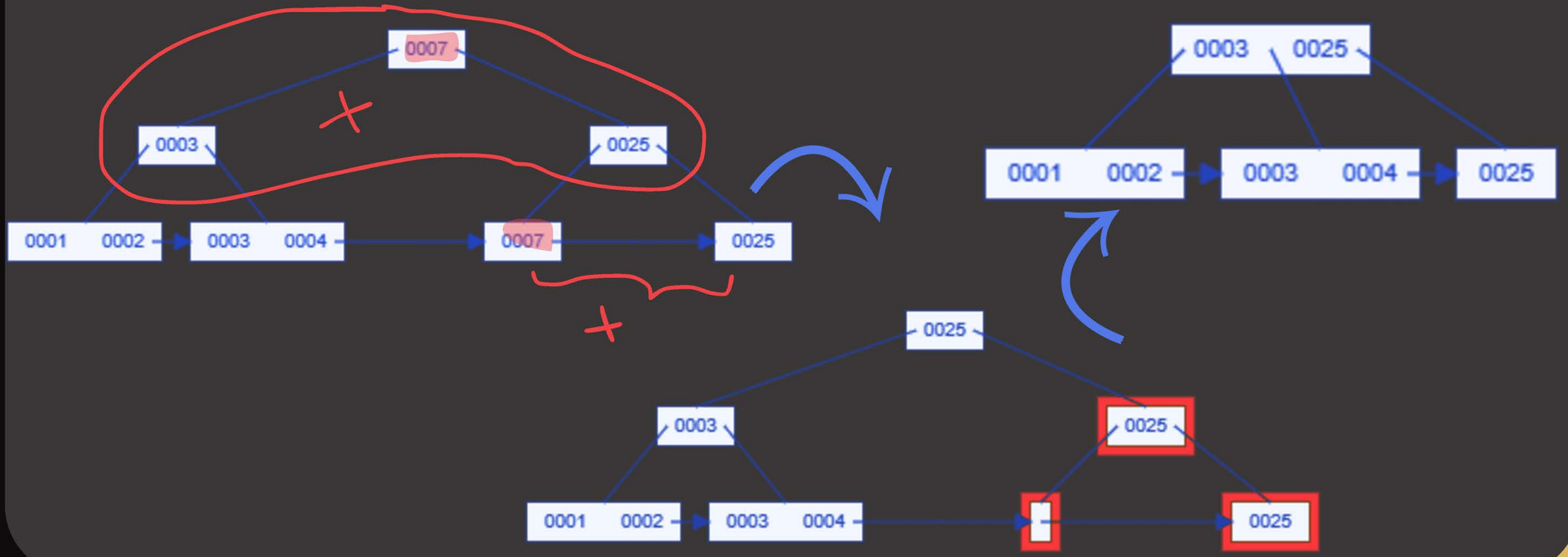
Case 5 and Case 4 conditions are the exact same - target leaf node, sibling node and parent node all have equal to “Min” number of keys allowed. The internal parent node will merge with it’s sibling and the tree height will shrink.

Case 5 (Shrinking the tree): Delete 25 - Find key in leaf node, remove and merge with sibling node like case 3. The internal parent node, will merge with one of its sibling node and their parent key.



B+ Tree (Deletion - Case 5)

Another Example: Delete 7: Exact same process as the previous example, however, since the key is also present in the root node which will be merged with it's children, the root node value is replaced with the lowest value from the right-most subtree first and then it is merged.

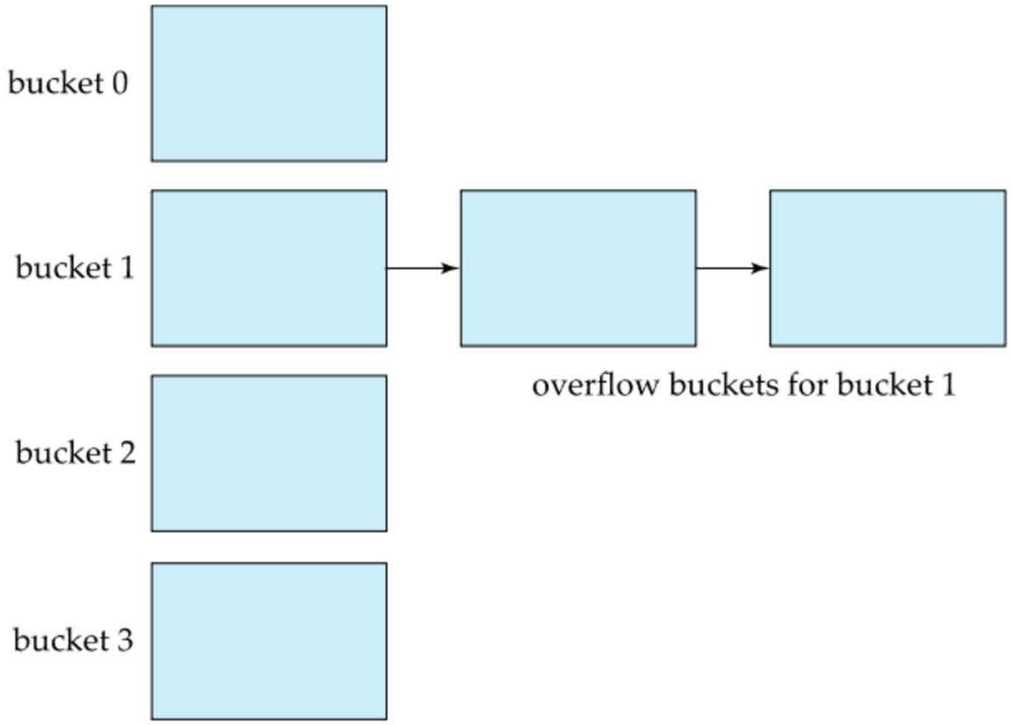


Static Hashing

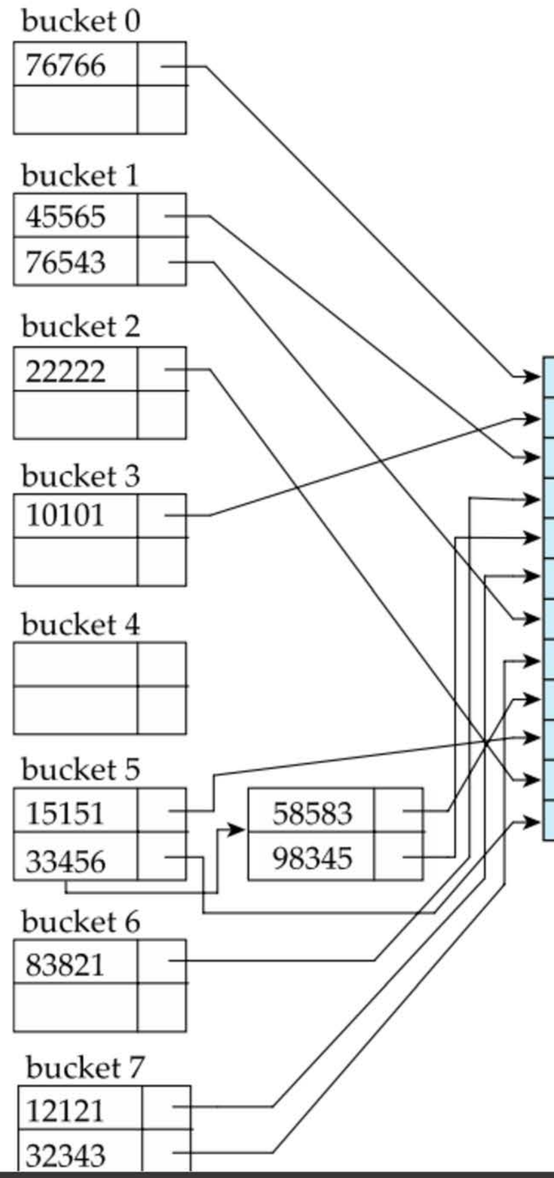
- Some RDBMS use hash tables to store indexes. One or more index entries are stored in a bucket (typically a disk block). We obtain the bucket of an entry from its search-key value using a hash function. Entries with different search-key values may be mapped to the same bucket; thus entire bucket has to be searched sequentially to locate an entry.

- Bucket overflow can occur because of insufficient buckets or skew in distribution of records. This can occur due to two reasons: multiple records have same search-key value or chosen hash function produces non-uniform distribution of key values.

- Although the probability of bucket overflow can be reduced, it cannot be eliminated; it is handled by using overflow buckets through the process of forward chaining/overflow chaining (overflow buckets chained together with a linked list). Other methods of resolving overflows is not suitable for database applications.



Hashing Example



Hash function,
h(ID) = (sum of digits in ID) % 8

Mod by 8 because, number of hash buckets is 8.

76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
33465	Gold	Physics	87000

hash index on *instructor,* on attribute *ID*

What Next?

LECTURE 8: TRANSACTIONS

LOADING.....